

Unit - 2

XML & JSON

XML

- Hi again! Good to see you back. Right, so you remember how we did forms in the previous unit? and we said that the data from these forms are sent to the web server in an *appropriate format*. We gonna talk about that appropriate format now
- XML was invented first, so we'll talk about that first.
- XML stands for Extensible Markup L. Why X? because it sounds cool.
- XML is basically just HTML but you define the tags.
- If you want to transmit data about 2 students in XML, the format would be like this:

```
<studentinfo>
  <student1>
    <name>Anurag</name>
    <age>18</age>
  </student1>
  <student2>
    <name>Cupcake</name>
    <age>5</age>
  </student2>
</studentinfo>
```

XML

- You receive or send this data in the form of strings. So we need to methods to:
 1. Convert an XML object to a string - `XMLSerializer()`
 2. Convert an incoming string to an XML object - `DOMParser()`

DOMParser

```
// assume that variable 'incomingString' is the string that contains the above XML. You need to
// parse that into a proper DOM object on which you can do .querySelector, .getElementById, etc.

let incomingString; // this will be the incomingString. How this will be defined is covered later in
the course.

var parser = new DOMParser(); // create the parser

var xmlDoc = parser.parseFromString(incomingString, "text/xml");

// this creates an xmlDoc. Now you can use this to access elements:

// now to print the name of the first student:
console.log(xmlDoc.querySelector("student1").childNodes[0].nodeValue)
```

JS

- You can see how even with this simple of a data, the query for getting the value is so large. That is why JSON was invented.
- Before moving on to JSON, we'll do the `XMLSerializer` as well first. I will be writing the code assuming that the `xmlDoc` is already defined. You want to send this doc over the internet. So this will be in the form of a string. To convert a doc object to a string:

XMLSerializer()

```
let serialiser = new XMLSerializer();

let stringToSend = serialiser.serializeToString(xmlDoc); // converts back xmlDoc to string
```

JS

JSON

- JSON stands for **J**avascript **O**bject **N**otation. It's pronounced as jay-son
- This format is basically just the same as a javascript object. If you represent the same student data as JSON, it would look like this:

```
{
  student1: {
    name: "Anurag",
    age: 18,
  },
  student2: {
    name: "cupcake",
    age: 5,
  }
}
```

- A million times more simpler, easier to read, and less text - which means you can transmit it faster.
- You also need two methods to do the same conversions:

1. JSON to string to send over the internet: `JSON.stringify()`
2. String to JSON: `JSON.parse()`

- You'll see that these methods are also simpler to use:

`JSON.parse()`

```
JS
let incomingString; // this is not an undefined or empty string. This is just a placeholder. assume
that this variable contains the string that just came in. The same string as the JSON structure
example above.

let parsedJson = JSON.parse(incomingString); // that's it. no parser needed, no nothing.

// Now parsedJson is just a JS object and you can access it just like any other object:
// to print student1's name:
console.log(parsedJson.student1.name); // makes way more sense than XML
```

`JSON.stringify()`

```
JS
// just like the name suggests, this is used to convert a parsedJson back to a string. aka string-
ify it. JS devs actually named it that.

let stringToSend = JSON.stringify(parsedJson); // that's one line to convert it back to a string

// stringToSend will be the stringified version of the json object. You can use this stringify
method on any JS object really.
```

Advanced HTML Elements: `audio` , `video` and `progress`

`audio`

- Back to HTML for a bit, This is how you embed some kind of audio in your website:

```
HTML
<audio>
  <source src = "/path/to/sound.mp3" type="audio/mp3" />
</audio>
```

There are various attributes you can appoint to the `audio` tag that defines how your audio will be played. For example, If I want my audio to have controls, and autoplay on page load, I'll give those attributes:

```
<audio autoplay controls>
  <source src = "/path/to/sound.mp3" type="audio/mp3" />
</audio>
```

You can also write this as:

```
<audio autoplay="autoplay" controls="controls">
  <source src = "/path/to/sound.mp3" type="audio/mp3" />
</audio>
```

- But this method just introduces redundant text.
- Here's a table of attributes you can use on the audio tag:

Attribute	Value	Description
autoplay	autoplay	Specifies that the audio will start playing as soon as it is ready.
controls	controls	Specifies that controls will be displayed, such as a play button.
loop	loop	Specifies that the audio will start playing again (looping) when it reaches the end
preload	preload	Specifies that the audio will be loaded at page load, and ready to run. Ignored if autoplay is present.
src	url	Specifies the URL of the audio to play

- You can skip the `source` tag inside and just give the `src` attribute to the audio tag as well:

```
<audio autoplay="autoplay" controls="controls" src = "/path/to/sound.mp3">
</audio>
```

- An advantage of using the source tag inside is that you can give multiple sources. Different web browsers support different formats of audio and video. For example, Chrome supports audio of both mp3 and OGG format, wherein Safari only supports MP3. By providing both mp3 and OGG formats, the browser will choose the first source it supports from the list. If it supports none of them, it'll show the text inside the audio tag:

```
<audio autoplay="autoplay" controls="controls">
  <source src = "/path/to/sound.ogg" type="audio/ogg" />
  <source src = "/path/to/sound.mp3" type="audio/mp3" />
  This text is displayed only if the above two sources fail
</audio>
```

video

- The idea of a video tag is the same as an audio tag, except here you specify the width and height of the video too. If you don't specify this, the video will take how much ever space it wants - aka in the resolution it was recorded in. This is often undesirable because you want the video size to behave according to your website.
- Here's how you define a video tag:

```
<video width = "320" height = "240" controls>
  <source src = "/path/to/video.mov" type = "video/mov"/>
  <source src = "/path/to/video.mp4" type = "video/mp4"/>
  Your browser doesn't support video tag. This text is only displayed when the above sources fail
</video>
```

And here are the attributes you can use with video tags:

Attribute	Value	Description
audio	muted	Defining the default state of the the audio. Currently, only "muted" is allowed
autoplay	autoplay	If present, then the video will start playing as soon as it is ready
controls	controls	If present, controls will be displayed, such as a play button
height	pixels	Sets the height of the video player
loop	loop	If present, the video will start over again, every time it is finished
poster	url	Specifies the URL of an image representing the video
preload	preload	If present, the video will be loaded at page load, and ready to run. Ignored if "autoplay" is present
src	url	The URL of the video to play
width	pixels	Sets the width of the video player

progress

- This is used to put a progress bar in your HTML.
- We usually use a `<label />` tag above or below the `<progress />` tag to tell what the progress is about
- Here's an example:

HTML

```
<!DOCTYPE html>
<html lang="en">

<head>
  <title>Progress </title>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link href="css/style.css" rel="stylesheet">
</head>

<body>
  <label for="progress-text">Upload Progress: </label>
  <progress value="30" max="100"></progress>
</body>

</html>
```



- You have to set the progress of this progress tag by modifying the `value` attribute of the `progress` tag.
- You do this in Javascript. You figure out the progress of whatever is happening through other methods and set the progress on the tag using JS.
- For example, I'll just make it go from 0 to 100 without any actual progress. There is nothing happening but I'm just gonna make the progress bar go from 0 to 100:

HTML

```
<!DOCTYPE html>
<html lang="en">

<head>
```



```

<title>Progress </title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<link href="css/style.css" rel="stylesheet">
</head>

<body>
  <label for="progress-text">Upload Progress: </label>
  <progress value="0" max="100"></progress>

  <script>
    var progress = document.querySelector("progress");
    var interval = setInterval(increaseProgress, 10);

    function increaseProgress() {
      progress.value++;
      if (progress.value ≥ 100) {
        progress.value = 0;
      }
    }
  </script>
</body>

</html>

```

- The `setInterval()` method of JS is used to do something repeatedly, in this case, call the `increaseProgress()` function repeatedly with a delay of 10 milliseconds. This repeating will only stop happening when I call `clearInterval(interval)` or when the window closes.
- You wouldn't be doing this in real life, you'd ofc get the progress value from somewhere else based on something that's actually happening, but this is just an example of how you can set the value of a `progress` tag with JS.

Canvas

- A canvas is like a real life canvas. You can draw stuff on it. Either 2D or 3D.
- Here's how you use a canvas:

```

<!DOCTYPE html>
<html lang="en">

<head>
  <title></title>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link href="css/style.css" rel="stylesheet">
</head>

<body>
  <canvas id="myCanvas" width="200" height="100"></canvas>

  <script>
    let canv = document.getElementById("myCanvas");
    let context = canv.getContext("2d");
    context.fillStyle = "#000000";
    context.fillRect(10, 10, 100, 100);
  </script>
</body>

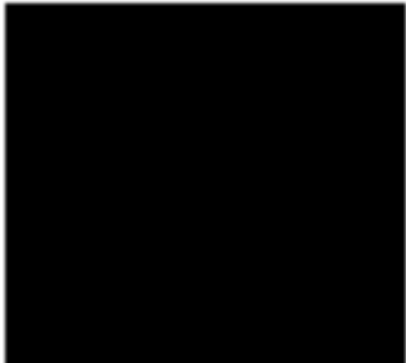
</html>

```

HTML

- First I make a canvas using the `<canvas>` HTML tag. Then I access it in JavaScript.

- The next line in JS is to get a `context`. This object contains context of what is already there in the canvas.
- We have to pass what kind of drawing you want to do on the canvas to the `getContext()` object, in this case, it's `2d`
- Then we use this context to draw various stuff.
- First, I set the `fillStyle` to the colour I want, I want black, who's Hex Code is `#000000`
- Then we use the `fillRect()` function to make a filled (solid) rectangle. The parameters we pass are the `(x,y)` values of the top left vertex of the rectangle and the width and height of the rectangle
- A you can see, this is super tedious, so nobody uses canvas to draw stuff manually like this. We have libraries to handle it.
- This is the end result:



There are a couple other functions available too and it's just listed out in a table. It wasn't taught in class completely either. So I don't think it's that important. You can just go through the table and roughly have an idea what it does just in case it comes in the exam as a one marker. In real life, we just google this kind of stuff. Nobody memorises this stuff

Method	Description
<code>fillRect(x, y, width, height)</code>	Draws a filled rectangle
<code>strokeRect(x, y, width, height)</code>	Draws a rectangular outline
<code>clearRect(x, y, width, height)</code>	Clears the specified rectangular area, making it fully transparent
<code>moveTo(x, y)</code>	Moves the pen to the coordinates specified by x and y
<code>lineTo(x, y)</code>	Draws a line from the current drawing position to the position specified by x and y
<code>arc(x, y, r, sAngle, eAngle, anticlockwise)</code>	Draws an arc centered at (x, y) with radius r starting at sAngle and ending at eAngle going anticlockwise (defaulting to clockwise).
<code>arcTo(x1, y1, x2, y2, radius)</code>	Draws an arc with the given control points and radius, connected to the previous point by a straight line

<code>createLinearGradient(x1, y1, x2, y2)</code>	Creates a linear gradient object with a starting point of (x1, y1) and an end point of (x2, y2).
<code>createRadialGradient(x1, y1, r1, x2, y2, r2)</code>	Creates a radial gradient. The parameters represent two circles, one with its center at (x1, y1) and a radius of r1, and the other with its center at (x2, y2) with a radius of r2.
<code>fillText(text, x, y [, maxWidth])</code>	Fills a given text at the given (x,y) position. Optionally with a maximum width to draw.
<code>strokeText(text, x, y [, maxWidth])</code>	Strokes a given text at the given (x,y) position. Optionally with a maximum width to draw.
<code>drawImage(image, x, y [,width, height])</code>	Draws the CanvasImageSource specified by the image parameter at the coordinates (x, y) with optional width and height

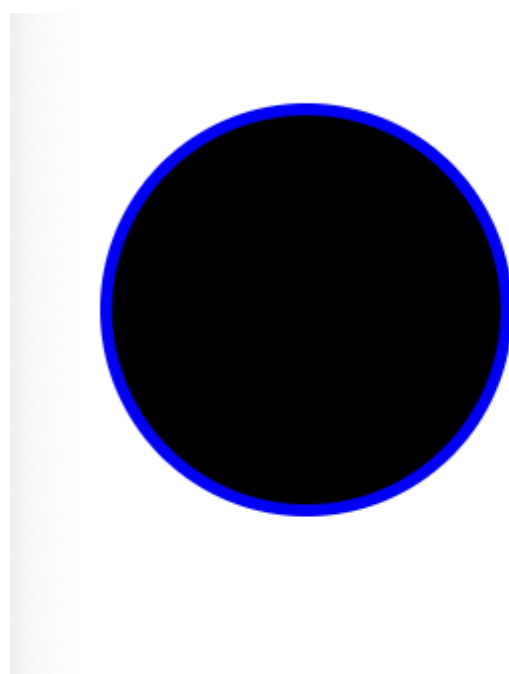
SVG

- This is kinda like an image format. But it's different from an image in every way except for the fact that it shows some kind of picture.
- Normal images contain data of what colour is there at what pixel. So when we zoom these images to a very large extent, they appear pixelated as you can define what colour is there for only a finite number of pixels.
- To solve this issue, we use SVGs (**S**calable **V**ector **G**raphics)
- As the name suggests, is kinda like a vector. You define the stroke (like you're drawing with a pencil) and how this stroke goes in the form of equations (vectors)
- The advantage of this is that your image would *appear the same quality no matter how huge you make it*.
- In the web, images are of various sizes and so are the user's monitors. Using a normal png image would make the image appear pixelated for people with huge monitors or, if your image is a small logo, and you want to render it big, it'll appear pixelated. So we use SVGs wherever possible.
- One disadvantage of this is that SVGs are good for simpler images, like logos. If you have a lot of detail in your images, like a person's profile picture, it's better to store it as a `png`.
- You define SVGs in the form of HTML tags. You can store this text in a `.svg` file and then use them as a normal image.
- There is more focus on the generating tags for the `svg` image in the slides. In real life, people use SVG generators to make SVGs. Nobody writes these tags manually in real life, but again, PES ☕
- To make an SVG, we start with an `<svg>` tag. Then we can put shapes inside it. There are a few predefined shapes available to draw, like `circle`, `rect`, `ellipse`, `polygon` and `text` too.
- Here's how you use them:

```

HTML
<svg>
  <circle cx="69" cy="69" r="50" stroke="blue" fill="black" stroke-width="3"></circle>
</svg>

```



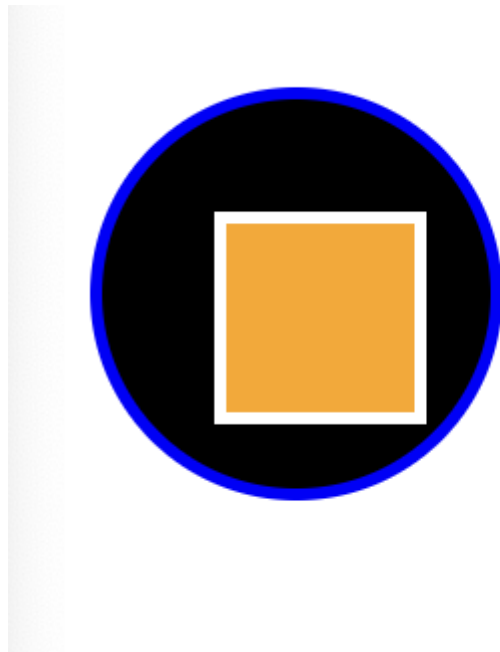
- The `cx` and `cy` attributes specify the x and y coords of the centre of the circle.

- You can give custom Hex codes too for **fill** and **stroke**.
- Now I'll draw a rectangle:

HTML

```
<svg>
  <circle cx="69" cy="69" r="50" stroke="blue" fill="black" stroke-width="3"></circle>
  <rect x="50" y="50" height="50" width="50" fill="orange" stroke="white" stroke-width="3"></rect>
</svg>
```

- The attributes here are pretty self explanatory



- Let's put one anda ● also:

HTML

```
<svg>
  <circle cx="69" cy="69" r="50" stroke="blue" fill="black" stroke-width="3"></circle>
  <rect x="50" y="50" height="50" width="50" fill="orange" stroke="white" stroke-width="3"></rect>
  <ellipse cx="30" cy="69" rx="30" ry="40" fill="purple" stroke="orange" stroke-width="2">
  </ellipse>
</svg>
```

- Time for a polygon:

HTML

```
<svg>
  <circle cx="69" cy="69" r="50" stroke="blue" fill="black" stroke-width="3"></circle>
  <rect x="50" y="50" height="50" width="50" fill="orange" stroke="white" stroke-width="3"></rect>
  <ellipse cx="30" cy="69" rx="30" ry="40" fill="purple" stroke="orange" stroke-width="2">
  </ellipse>

  <polygon points="200,10 250,190 160,210" fill="black"></polygon>
</svg>
```

- The **points** attribute is a list of space separated **x,y**s



- Adding text:

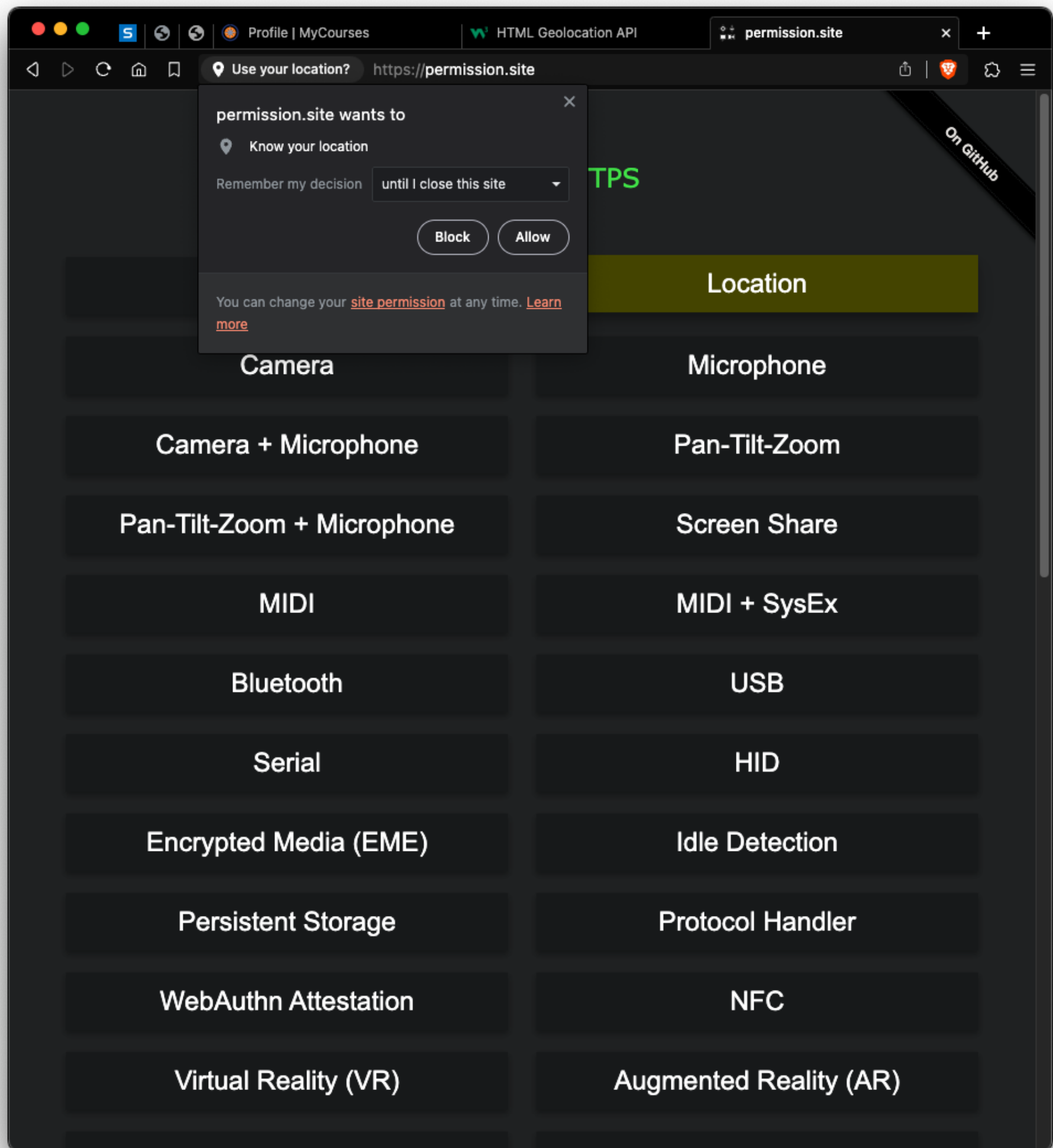
HTML

```
<svg>
  <circle cx="69" cy="69" r="50" stroke="blue" fill="black" stroke-width="3"></circle>
  <rect x="50" y="50" height="50" width="50" fill="orange" stroke="white" stroke-width="3"></rect>
  <ellipse cx="30" cy="69" rx="30" ry="40" fill="purple" stroke="orange" stroke-width="2">
</ellipse>
  <polygon points="200,10 250,190 160,210" fill="black"></polygon>
  <text x="20" y="25" fill="#FFD700" transform="rotate(30 20 100)">hello</text>
</svg>
```

- The transform property can be either **rotate** or **translate** or **skewX** or **scale**. I used to rotate property. It's used to rotate a certain degrees about a point. Here I am rotating **30**° about **20,100**

The Geolocation API

- This is used to ~~stalk~~ locate your users.
- Imagine you're building a food delivery website. You gotta use this API to locate the user to decide the delivery address.
- When you see those 'wants to use your location' pop ups, this is what is being used by the website:



- You can head over to permissions.site and click the location button to see how it looks like on your browser. This site is just used to dummy test permissions.
- Anyways, this is how you implement it:

```

<!DOCTYPE html>
<html lang="en">

<head>
  <title>Ima stalk you</title>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
</head>

<body>
  <div class="location"></div>
  <!-- This div is used to display the location we found out, we gonna put the latitude and longitude positions here -->

```

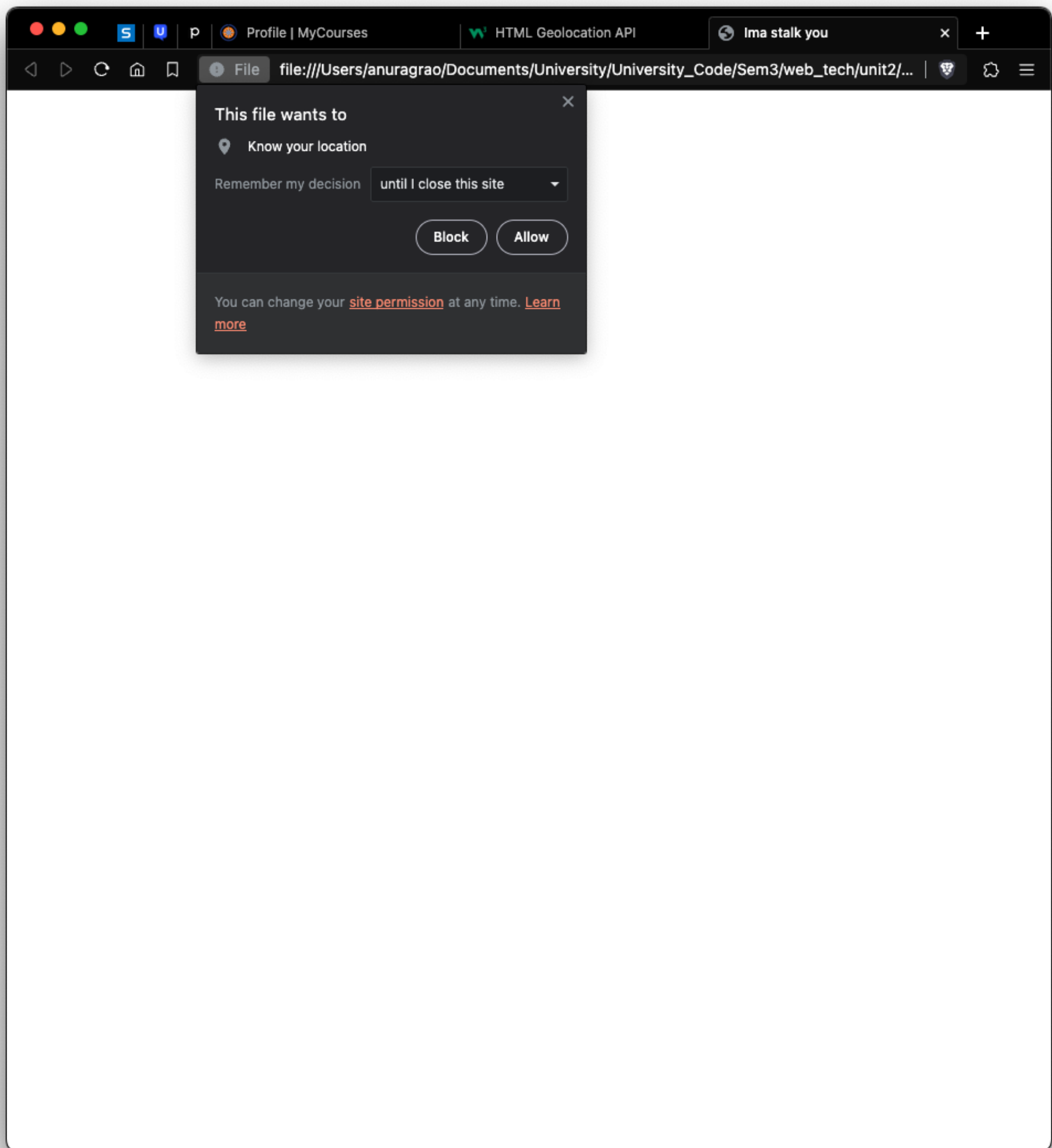
```
<script>
  let div_position = document.querySelector(".location"); // select using class

  // function that sets the divs contents on successfully getting the geolocation
  // the 'position' parameter is an object that's passed into this function by the getGeoLocation
  function. It contains information like coords, accuracy, altitude, altitudeAccuracy, heading, speed
  etc.
  function show_position(position) {
    div_position.innerHTML = `Latitude: ${position.coords.latitude} Longitude:
    ${position.coords.longitude}`;
  }

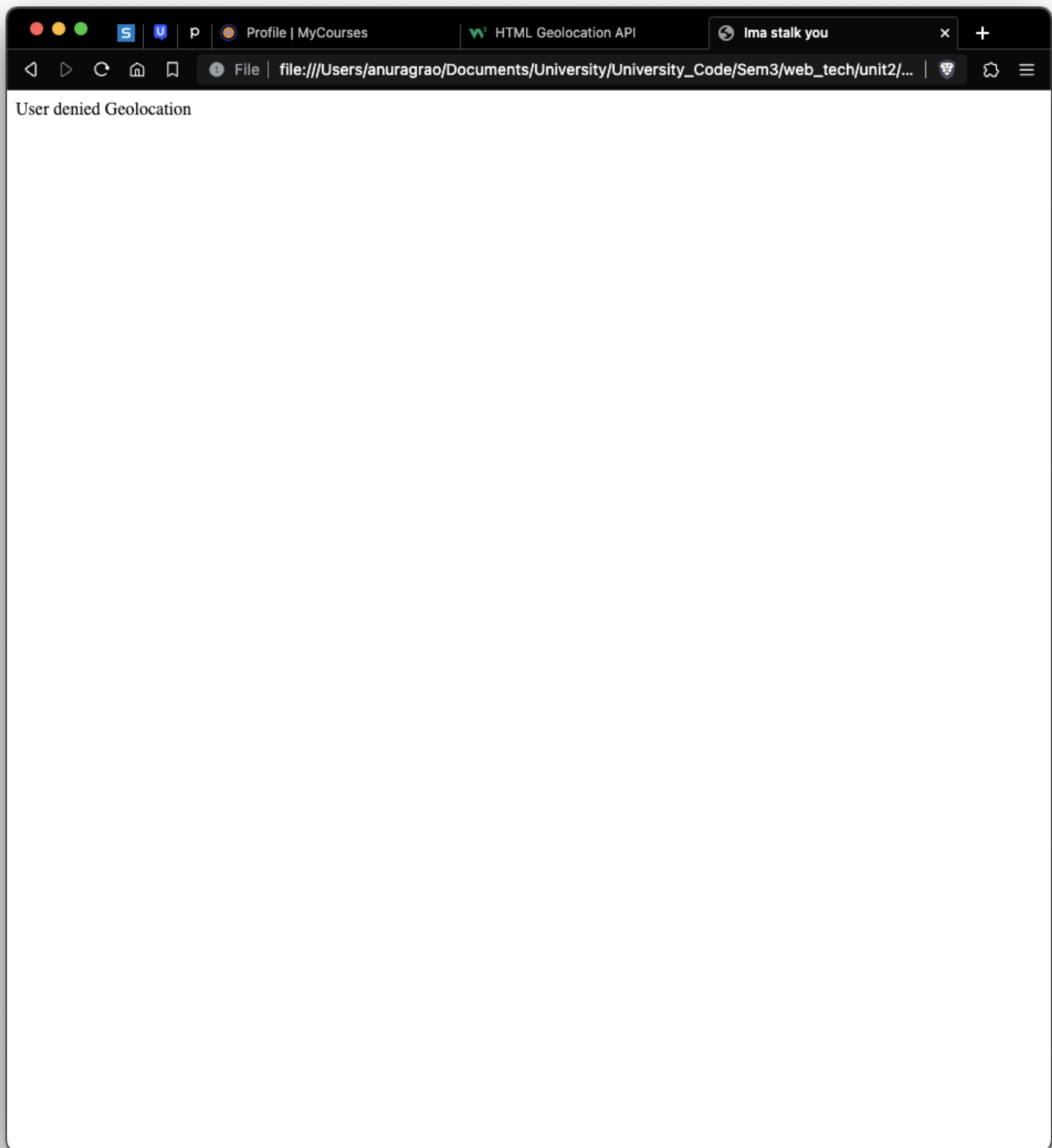
  // if you're confused why I used backticks (``) to enclose my string and wtf that syntax is: That's
  called a template literal. You enclose your string in backticks instead of quotes and you can use
  variables or expressions inside your string by putting them inside ${} like I did in that string. It
  helps you avoid complex concatenations that may be confusing.

  // function to set contents of the div to a failed message incase we fail to get location
  // this error functions accepts the 'error' object that contains a short code and a message. The
  code is encoded this way:
  // 1→ permission denied
  // 2 → position unvailable
  // 3 → timeout
  function location_failed(error) {
    div_position.innerHTML = error.message;
  }

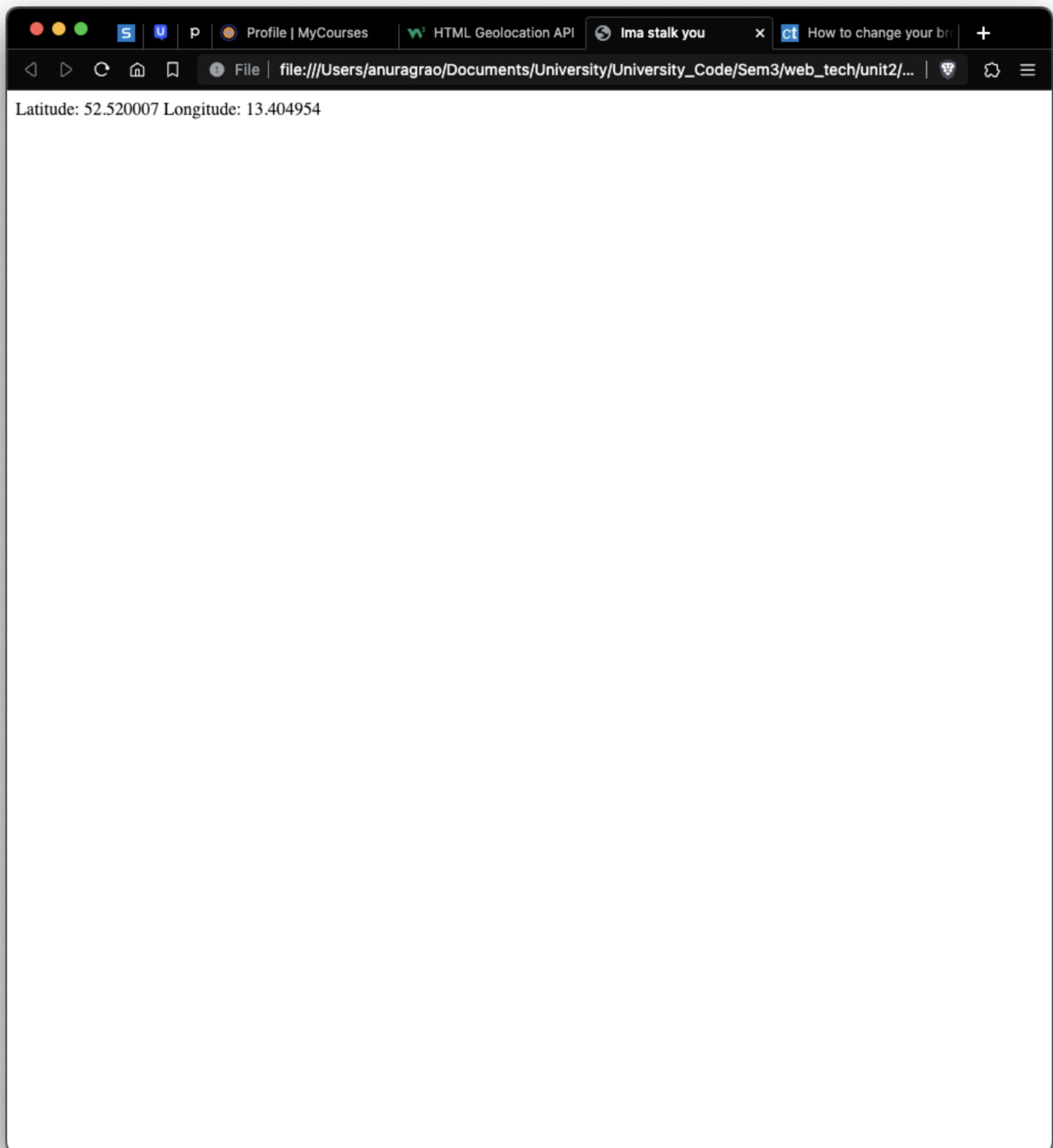
  // we use the navigator object and call it's geolocation.getCurrentGeolocation function. This
  function takes in two callback functions and an optional geo location options object. The first one
  is called with the 'position' object parameter if we successfully get location. If we fail to get
  the location, the second function is called. Passing the second error function is also optional In
  our case, the first function aka success function gets the position and sets it to the divs
  innerHTML. On failure, we give that failure message.
  navigator.geolocation.getCurrentPosition(show_position, location_failed);
</script>
</body>
</html>
```



- If I click on 'block':



- But if I click 'allow':



- That's a fake location, don't try to look that up- That's not where I live.
- You can also use the `watchPosition()` function to return the live location of a user. It returns the new location every time the location of the user changes. This function also accepts parameters similar to `getCurrentPosition()`

Web Workers

- JavaScript by nature is a single threaded language.
- You know how your CPU has 'threads' right. Each individual thread can process one thing at a time. More the threads, more things you can do at once and the faster your CPU is.
- Programming languages can make use of these threads to execute. By `single threaded`, we mean that a single JS program can run on only and only 1 thread.
- But if you want to make your program faster, what do you do? You divide your program's job into multiple pieces and spawn a new JS program for every single piece. Yeah that's a very duct taped solution, and most devs don't like web workers but it is what it is. There are other ways to make your programs run faster though.
- Anyways, back to web workers, making use of these allows you to run multiple programs at once - aka asynchronously and they are independent of each other - aka autonomously.
- This is usually used when you have to make API calls to another server and request data from it. It can take sometime for this data to come back and you don't want to block the main thread waiting for the other server to reply. It's kinda like texting

your crush and doing absolutely nothing until she/he replies. That's a waste of time.

Here's how you implement them:

This will be your main HTML file:

```
HTML

<!DOCTYPE html>
<html lang="en">

<head>
  <title>Web Workers Demo</title>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
</head>

<body>

  <script>
    const message_div = document.querySelector(".message-from-worker");

    let worker = new Worker("./worker.js"); // you pass in the address to your worker.js file. This
    is just the partner JS file that you'd use as a worker to do things for you parallely.

    // The worker will do things for you but how do you communicate with it? With messages.
    // let's send a message to the worker
    worker.postMessage('Hello, worker!');

    // What do we do when we receive a message from the worker:
    worker.onmessage = function (message) {
      console.log("received reply from worker: " + message.data);
    }
  </script>
</body>

</html>
```

And here's `worker.js`:

```
JS

// on message to the worker, we console log it and send a reply using postMessage
self.onmessage = function(event) {
  console.log("got message from main thread: " + event.data);
  self.postMessage("Hello! This is worker. got your message!");
}
```

- The `self` object is used to refer to global scope. It is kinda like `this` but `this` refers to the scope that the function it was called from. `self` always returns the global scope no matter what. Here, we are using the global scope of the file to use the `postMessage` attribute.
- Also, chrome does not allow web workers from local files. All this while, we were just opening our html file in the browser, just like any other file. But this isn't how it's supposed to be done. You should ideally have a server running on your system that serves these files appropriately. This prevents unauthorised access to your files from the internet.
- The reason why chrome doesn't allow local files to execute web workers is because of security. When you open a HTML file locally, it has the ability to access any file on your local system. This isn't good, so chrome blocks it. Not only chrome but most modern browsers block this.
- So we need to setup a proper server on our local machine (aka your computer)
- The official college way to do it is to install the `XAMPP` server. `XAMPP` is a software that allows you to quickly spin up a combination of servers, including a database server (mysql) and a an `Apache web server`.
- You can use the `XAMPP` method but there's an easier method available. There's an extension on vscode called `Live Server`. (This isn't available if you're using the OSS version of vscode, because Microsoft ☹️)

- Once you install this extension, you can spin up a live server from vscode itself. You'll see a button at the bottom of your editor that would say 'go live'. Clicking on it starts your server.
 - It should automatically open the browser tab for you, if not, you can go to `http://127.0.0.1:5500` on your browser.
- `127.0.0.1` is the default address in every computer that refers to itself, aka home. That is why you might have seen t-shirts like this:



Anyways, back to web workers, when I open that html file we created, this is what we see on the browser's console:

```
got message from main thread: Hello, worker! worker.js:2
received reply from worker: Hello! web_worker.html:25
This is worker. got your message!
>
```

- **NOTE:** The web worker file does not have access to the `document` object of the browser. This means that it *can not* perform DOM manipulations.
- In the real world, you wouldn't be sending Hi and Hellos to your worker, you'd most likely be sending JSON objects for communication. (Remember `JSON.parse` and `JSON.stringify`, here's a use case for it)

jQuery

- This is a way of making selecting an HTML tag from JS easier.
- We saw in the last chapter how to get a reference to an element in HTML:

```
document.getElementsByClassName("someclass");
// or
document.getElementsByName("somename");
// or
document.getElementById("someid");
// or
document.querySelector("someselector");
```

- This is too much effort. jQuery makes this easier.
- jQuery is a "write less do more" **JavaScript library**. It is NOT another language!
- It not only makes selection easier, but it combines and makes a one liner out of a task that might take multiple JS lines.
- It wraps them into methods that you can call with a single line of code.

Setting Up jQuery

- Again, JS being JS, gives you multiple ways of doing the same thing
- You can download the jQuery script file locally and link it with a script tag:

```
<script src="path/to/jquery.js"/>
```

- OR, you can use this thing called a CDN.
- A CDN stands for a Content Delivery Network. It's a web link from where your JS file is downloaded and imported from. This is advantageous because a CDN provides multiple servers.
- If you serve your website in a single server, in India, then a person who's trying to access your website in say, Canada would have a bad experience since the main server is so far away from him.
- On a CDN, you have multiple servers across the world and the one closest to you is used when you access a website. This makes your website load faster.
- You can get CDN links to jQuery on the official site [here](#)
- Once you get a link, you include it in a script tag like this:

```
<script src="https://code.jquery.com/jquery-3.7.1.min.js"
  integrity="sha256-/JqT3SqfawRcv/BIHPThkBvs00EvtFFmqPF/LYI/Cxo=" crossorigin="anonymous">
</script>
```

// the integrity property is to make sure the script wasn't accidentally tampered with on the way from jquery's servers to yours. In our case, it makes use of a sha256 checksum to do it. checksums are really interesting, you can google about it if you have time.

// the integrity property is optional. The crossorigin attribute specifies if we use credentials or not. This is also beyond the scope of this chapter and is more about authentication and networks. It basically specifies if the CSN request is made with user credentials or not. In our case, it is not, since it's 'anonymous'. If the crossorigin attribute is specified, the server also sends error information if any error occurs. That makes it easy for us to handle this error in our code. I understand this is very vague but we'll need an entire other section to explain this properly and I'm pretty sure you don't have the time for that. Just google it mate.

- You can also use a package manager like **npm**, **pnpm**, **yarn** etc but that's for **NodeJS**. We mostly won't be using this until we get to **NodeJS**

Getting Started

```
<!DOCTYPE html>
<html lang="en">

<head>
  <title>Basic jQuery</title>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link href="css/style.css" rel="stylesheet">
```

```

<!-- import jquery -->
<script src="https://code.jquery.com/jquery-3.7.1.min.js"
  integrity="sha256-/JqT3SQfawRcv/BIHPThkBvs00EvtFFmqPF/LYI/Cxo=" crossorigin="anonymous">
</script>
</head>

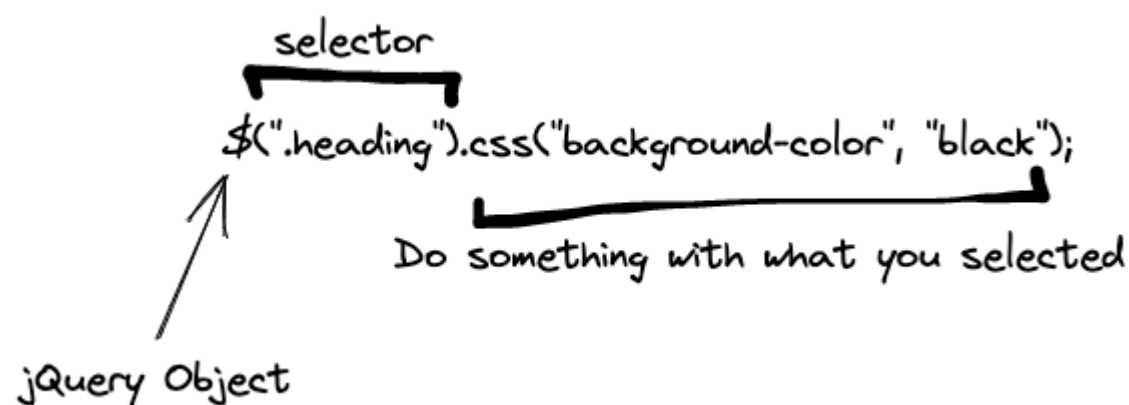
<body>
  <h1 class="heading">This is a H1 Tag. I am going to set the color of this to be white and
background to be black in
  jquery</h1>

  <script>
    $(".heading").css("color", "white");
    // $() is shorthand for the core jQuery function.
    // You can write this function as jQuery() too
    $(".heading").css("background-color", "black");
  </script>
</body>
</html>

```

This is a H1 Tag. I am going to set the color of this to be white and background to be black in jquery

The basic structure of a JQuery statement looks something like this:



- These selectors are the same selectors you'd use in CSS.

jQuery Selectors

- You can use just a CSS selector in the jQuery selector parameter. Look at the picture above for what the 'selector' is.
- In addition to this, you can use a few other specialised selectors:

Selector	Explanation
Combination	You can select a combination of elements by using multiple selectors. These multiple selectors are just space separated. For example, If I wanted to select two buttons, one with class button1 and the other with button2 , I can do it with <code>\$(".button1 .button2")</code>
Basic Filters	Imagine you're selecting multiple elements with one single class but you only want to select even elements or odd elements or only the first or last element from those multiple elements, you can do it with the <code>:first</code> , <code>:last</code> , <code>:even</code> and <code>:odd</code> selectors. For example, If I have multiple li elements and I want to select every even child, I can do it with <code>\$("li:even")</code> . This is especially useful when you have a list or a table and you want the alternate rows to be coloured different.
Content Filters	This is similar to the filters I explained above but it filters based on what is there inside the element. It can either be <code>:empty</code> if you want to select only empty tags, or <code>:has(text)</code> if you want to select only ones that have a particular text, or <code>:has(selector)</code> if you want to target elements which have a particular element inside them (which you target

Selector	Explanation
	with a selector). The usage would be the same as above, for example <code>\$("#li:has('name'))</code> . Useful when I want to only change properties of a particular field in the list
Attribute Filters	These filters are used to filter by an attribute of a particular HTML. To remind you in case you've forgotten, an <i>attribute</i> is what you put in those HTML tags. For example <code><input whatever_is_here_is_an_attribute></input></code> . You can filter by attribute by using the <code>[attribute]</code> , <code>[attribute=value]</code> and <code>[attribute!=value]</code> selectors. For example, If I want to select the input tags that are checkboxes, I can do <code>\$("#input[type='checkbox']")</code>
Form Filters	You can use filters like <code>:input</code> , <code>:text</code> , <code>:submit</code> , <code>:password</code> , <code>:enabled</code> , <code>:checked</code> to filter and select elements inside a form. For example, If I wanted to find only the submit button out of all the buttons on the form, I'd use the <code>\$("#button:submit")</code> selector.

jQuery Methods

- Now once you have selected these elements, you have to do something with them-
- jQuery methods define what methods you can use on these selected methods.
- If you see the structure of a jQuery statement, there's the selector part and the 'do something with what you selected' part. This section is about this 'do something' part. You'll need to chain this with a selector to make a full jQuery statement.
- Here is a general list of methods:

Methods	Explanation
DOM Manipulation Methods	<code>before()</code> , <code>after()</code> , <code>append()</code> , <code>appendTo()</code> are used to do DOM Manipulation. <code>append()</code> and <code>appendTo()</code> do the same thing but there's a difference in syntax. When you do <code>\$("#foo").append("bar")</code> , it appends <code>bar</code> to <code>foo</code> . When you do <code>\$("#foo").appendTo("bar")</code> , it appends <code>foo</code> to <code>bar</code> . Here <code>foo</code> and <code>bar</code> could be a jQuery object or an actual HTML tag. For example, <code>\$("#<div>Hi</div>").appendTo(\$(".some_class"))</code> is still valid. It will append that div we created to whatever is selected by <code>.some_class</code> . <code>after()</code> and <code>before()</code> behave similarly. When you do <code>\$(".some_class").after("<div>Hi!</div>")</code> , it will append after the element with <code>.some_class</code> . Please note that it isn't necessary you have to create a new element too. If you use a jQuery object and append that, the element will be removed from it's previous position and put in it's new position. <code>\$("h2").appendTo(\$(".container"));</code> will append all <code>h2</code> s in the document to whatever <code>.container</code> selects
Attributes	<code>.addClass()</code> , <code>.css()</code> , <code>.attr()</code> (attribute), <code>.html()</code> , <code>.val()</code> , <code>.text()</code> can be used to get the attributes or modify the attributes of an element. The names are all self explanatory. For example, If I wanted to set the width of all images to be 300px, I'd do <code>\$("#img").attr("width", "300")</code>
Events	This would be the events like click, hover etc. The <code>click()</code> , <code>.on()</code> methods etc take care of this. More about this in the upcoming section
Effects	You can add effects and animations like <code>fadeOut()</code> , <code>animate()</code> , <code>toggle()</code> etc. More about this too in upcoming sections
AJAX	You can use this to send AJAX requests, more about these in the upcoming section. The methods we'd use would be <code>.load()</code> , <code>.get()</code> , <code>.ajax()</code> , <code>.post()</code> and <code>.getJSON()</code>

- Attributes like `.css()`, `.attr()`, `.val()` etc behave different depending on the parameters passed to them
- They can either be used to get or set an attribute. For example, If I had to get the background colour of a button, I can do:

```
JS
let bg_color = $(".button").css("background-color"); // gets the background color of .button and assigns it to bg_color
```

- Now if I pass 2 attributes to the `.css()` method:

```
JS
$(".button").css("background-color", "black"); // sets background color to .button
```

- We can also chain methods together like this:

```
JS
$(".button").css("background-color", "black").css("color", "white").html("<div>this sets inner html</div>");
```

- I have changed the background to black, font color to white, and added an inner html as a div that says 'this sets inner html'
- This is possible because most jQuery methods return the object itself. (It's kinda like in DSA when we returned the new head of a singly linked list) However, some jQuery methods **do not** return the object itself but some value. For example, If we try to get

the background-color again: `$(".button").css("background-color")`, this would return us the hex code of the background color and not the object itself. We cannot chain this further.

- Similarly, `.val()` also doesn't return the object itself. It is mainly used to get the value of form inputs. So this won't work: `$("#input[type = 'text']").val().css("background-color", "black");` because `.val()` doesn't return an object.

Events And Effects In jQuery

- An event is a any kind of interaction on your website. We did this before in pure JS. Now we'll do this using the jQuery library. Again, jQuery isn't a whole other programming language but just a Javascript library. Many people have a confusion at the beginning that it's a whole other language (this is mainly because of it being called *jQuery*)
- To attach an event listener to an element, you can use the `.on()` method. This method accepts two parameters.
- The first one is the event name, in the form of a string. For example `"click"` or `"pointermove"` or `"hover"`
- The second one is a callback function that will be called when that event is triggered. It works exactly like the event listener method we did earlier. This function is called an event listener here too
- Here's an example:

```
function send_alert(){
    alert("Clicked!");
}
$(".button").on("click", send_alert);
```

JS

You can also define a function on spot (anonymous function), and use jQuery statements inside it too:

```
$(".button").on("click", function () {
    $("body").css("background-color", "black"); // sets the background color of body to be black
    every time the button is pressed
});
```

JS

An example showed in every class for this is using the `slideToggle()` method to slide in and slide out a text on a button click. Here's how that's done:

```
$(".button").on("click", function() { // .button is a button defined in html with a class 'button'
    $(".text").slideToggle(); // .text is a div defined in html with a class 'text'
})
```

JS

- You can also define custom animations with the `.animate()` method:

```
$(".button").on("click", function() {
    $(".text").animate({
        fontSize: "4rem"; // rem is a relative unit for size.
    }, 1000)
})
```

JS

- The `animate` function takes in two parameters, the first is an object that has the css styles. Remember these styles must be in camelCase.
- The normal CSS names go like this `background-color` if they have two words in them. Unfortunately in JS, that `-` would be treated as a minus. So the same CSS property names are in camelCase in JS.
- The second parameter is the amount of milliseconds the animation must last. More this number, slower your animation.
- Other animation methods are `hide()`, `fadeOut()`, `toggle()` etc. It's used to hide the element, or fade it out, etc. You get it. The names are self explanatory.

Note: You can use the `this` keyword in AJAX too. `this` just refers to the scope or the object where a particular function was called from. Imagine if I had to click a text and on clicking the text, it disappears, I can code it up like this:

```
$(".text").on("click", function () {
    $(this).hide() // here, 'this' will refer to the DOM .text element. So I don't need to give
    $(".text") again.
})
```

JS

Callbacks And Promises

- A callback is basically a function that you pass to another function as a parameter. We used callbacks in `setInterval`, in the jQuery `.on()` method as well. The event listener we passed was a callback function.
- A promise is a new concept though.

A Promise

- Imagine a function that takes sometime to complete.
- This function would usually be an API call to your backend. It would take sometime to talk to the backend server and get the data back from backend. This time maybe very little or a lot based on the internet connection.
- So imagine a function like this:

```
let db_reply = get_db_reply();
```

JS

`get_db_reply()` will send an API request to the database and we are assigning the reply to a variable called `db_reply`.

- We talked earlier about how JavaScript was a single threaded application. So only one thing can happen at a time.
- When this line is executed, it is just waiting for the `db_reply` to come through. This can take a few milliseconds to about 30 seconds too if you have bad internet.
- And while this is happening, your entire website would be unresponsive.
- Now imagine if we had to fetch data from multiple sources step by step. And fetching data from one source depended on the other completing first, then we'll need to implement it using callback. The moment the first function finishes, then the second has to run.
- You might think why not just call the functions one after the other. It would work if all the functions took a similar amount of time. If I wanted to run `func2()` only after `func1()` because I'm using the data returned by `func1()` in `func2()`, I will need to make `func2()` wait for `func1()` to finish, but `func1()` could take it's own sweet time to finish. I could have used that time for more productive stuff.
- Anyways, so when we have situations like that, we use callbacks. We pass in the dependent function as a callback to the host function. In our example, `func1()` will have a callback of `func2()` as `func1(func2)`
- Now imagine if there was a `func3`, and a `func4` and so on depending on each other, that would lead to multiple layers of callbacks. This would make your code difficult to read.

```
getData(function(a){
  getMoreData(a, function(b){
    getEvenMoreData(b, function(c){
      console.log('Callback Hell: ', c);
    });
  });
});
```

JS

- You see how this nested callback thing gets more and more confusing as the layers grow.
- We use `promises` to resolve these issues. A `promise` is kinda like a promise in real life. The function promises you to return the value in the future and tells you to continue executing.
- Then at some point in the future, it can either keep the promise and return the value you expected from it, aka `resolve` the promise
- Or it'll break the promise, and `reject` the promise. This happens when something in your database call got messed up.
- A promise is just another object in JS too- ofc.
- You construct it using the `new Promise()` constructor like this:

```
let promise = new Promise(function (resolve, reject){
  // some asynchronous operation here
  if (/* operation successful */) {
    resolve(value);
  } else {
    reject(reason);
  }
});
```

JS

- This is often implemented with this thing called an *arrow function*. An arrow function is another way of declaring functions in JS. This is how you do it:

```
const func1 = (list_of_parameters) => {
  // function code
}
```

since we do `() => {}` to create the function, it's called an arrow function.

So, I can rewrite the promise code to be like this:

```
let promise = new Promise((resolve, reject) => {
  // some asynchronous operation here
  if (/* operation successful */) {
    resolve(value);
  } else {
    reject(reason);
  }
});
```

- The asynchronous operation is usually a database call.

The example given in the slides is about getting weather information from an API. They didn't elaborate much on the promise chaining part. But I will. That's the whole point of using promises and you'd think it's pointless if you don't understand promise chaining.

- Imagine there's a weather API that either tells you that the weather in your place is good or bad.
- Next, if the weather is good, you hit another API to tell you the nearest icecream place.
- Next, you try to book the nearest icecream for an icecream date with your partner.

Here is how you'd implement it using promises:

```
let getWeather = new Promise((resolve, reject) => {
  // assume that the weather is good and the API returns true
  let niceWeather = true;
  if (niceWeather) {
    resolve(true);
  }
  else {
    reject("Bad weather :(");
  }
});
```

```
let icecream_date = getWeather
  .then((weather) => {
    console.log("Weather is nice!")
  })
  .catch((error) => {
    console.log(error);
  })
  );
```

- This might be a lot to take in but let's go step by step. Here's how promises work:
 1. You create a new promise for a task you want to do
 2. The task starts executing asynchronously as soon as you create the promise. It does not block the main thread but runs in parallel. How does it run in parallel when JS is single threaded is a whole other topic. This topic is already getting long so I'm not gonna explain that nor is it in syllabus. The brief explanation is that it offloads tasks like this into the browser's APIs which allows the main thread to continue executing.
 3. The promise takes in a callback function. This callback function has two more callbacks, `resolve` and `reject`
 4. The function you passed into the promise is responsible for doing your asynchronous operations. Then call `resolve` or `reject` depending on the result of your operation. `resolve()` if you get a value and you kept your promise, `reject()` if you broke your promise

5. Now since the code had started running when you made your promise only, it keeps running in the background until a resolve or reject is reached and stores it.
6. This is the second part of promises - actually using them:
7. You do `promise_object.then()` This `then()` function takes in another callback function that accepts the resolved variable from the promise. In our case, the promise is about weather is good or not. We resolve it's good and return true. The `then` function has a callback function inside that accepts this true value.
8. The summary is that, the function you pass to the `then()` function runs when your promise resolves and returns a value. Your function has to handle this value so that's why it accepts it as a parameter.
9. After the `.then()` function, we also need to do a `.catch()` . `.catch()` also takes a callback function. This function handles the error text. This need not be a text but it can be anything, any object. I just kept it as a text for simplicity. Inside the catch function, I'm just logging the `reject` text
10. You can also have a `.finally()` block that will run finally regardless of if the promise resolved or rejected. It's kinda like `try` , `catch` and `finally` in C.

Now time for the chaining. I am going to extend that program and add two more promises to it:

```
JS
let getWeather = new Promise((resolve, reject) => {
  // assume that the weather is good and the API returns true
  let niceWeather = true;
  if (niceWeather) {
    resolve(true);
  }
  else {
    reject("Bad weather :(");
  }
});

let getIcecreamPlace = new Promise((resolve, reject) => {
  // this would also come from an API. Assuming that the API returns the nearest icecream place now
  let nearest_icecream_place = {
    name: "Naturals Icecreams",
    distance: 5,
  }

  if (nearest_icecream_place) {
    resolve(nearest_icecream_place);
  } else {
    reject("Icecream Place Not Found :(")
  }
});

let getBooking = new Promise((resolve, reject) => {
  // your booking confirmation will also come from an external API. Assuming you got a booking here
  let booking = {
    name1: "Anurag",
    name2: "Cupcake",
    time: "1600",
  };

  if (booking) {
    resolve(booking);
  }
  else {
    reject("Got no booking :(")
  }
});
```



```

let icecream_date = getWeather
  .then((weather) => {
    console.log("Weather is nice!")
    return getIcecreamPlace;
  })
  .then((IcecreamPlace) => {
    console.log("Got icecream place!");
    console.log(IcecreamPlace);
    return getBooking;
  })
  .then((booking) => {
    console.log("Got booking!");
    console.log(booking);
  })
  .catch((error) => {
    console.log(error);
  })
);

```

- On every `then()` stage, we just say that that particular stage is a success and then return the next promise. Then the chain grows. The last thing on the chain is the bookings one which gives you the booking data.
- The output of the above code will be:

```

Weather is nice!
Got icecream place!
{ name: 'Naturals Icecreams', distance: 5 }
Got booking!
{ name1: 'Anurag', name2: 'Cupcake', time: '1600' }

```

Now if I screw up the `getIcecreamPlace` promise and set the `nearest_icecream_place` to null:

```

let getIcecreamPlace = new Promise((resolve, reject) => {
  // Making the API now return null. Something went wrong with API Call
  let nearest_icecream_place = null;

  if (nearest_icecream_place) {
    resolve(nearest_icecream_place);
  } else {
    reject("Icecream Place Not Found :(") // This is called
  }
});

```

JS

Now moment the `getIcecreamPlace` stage `rejects` it's promise, the `.catch()` is triggered and it prints that the icecream place was not found:

```

Weather is nice!
Icecream Place Not Found :(

```

The whole point of this promise is that it helps avoiding passing a callback inside a callback inside a callback. This is how we avoided it. If I had to implement the same thing inside a callback, I'd call `getBooking` first, which would have a callback of `getIcecreamPlace`, which would have a callback of `getWeather`. This would make our code difficult to read and maintain.

Single Page Application (SPA)

- We build HTML files until now right. These are all a single page.
- But in a real world web application, you'd have multiple pages. One page for login, one for home page, one for accounts page etc etc.
- And we can just make different pages for these and link them using `<a href="link/to/next/page">Click to go to next page</a>` tag. (`<a>` tag is an anchor tag. It's used to redirect the user from one page to another)

- This would cause a lot of page reloads when the user is going around the app. That's not a really nice thing especially if your user has a bad internet connection. Every page would take ages to load.
- Instead, we can make one page. And hack together a way for the page contents only to change based on interactions.
- This way, we avoid redirects. But a disadvantage is that we'll need to use a lot of JavaScript to achieve this. This is called a single page application. You'd have seen this in most web apps like gmail or instagram. When you go to the DM's tab from your feed, the entire page does not reload. Only the contents of the page changes.
- A naive way to do this would be to use an `iframe` tag and change the `src` attribute of it depending on user interaction. This isn't ideal and would get confusing very soon. A better way to do this is using the *XHR object*
- In real life, almost nobody uses XHR but we have frameworks like React, Svelte, etc to make this way simpler.

The XHR Object

- **XHR** stands for **XML HTTP Request**.
 - This object is used to asynchronously get the new XML or HTTP data from the backend server. This enables you to fetch data without any page refreshes.
 - This helps you in making your website truly single page.
 - This whole method of asynchronously fetching data and setting it to our HTML is called **Asynchronous Javascript and Xml** is called **AJAX**.
 - XHR object has various methods and properties that we use.
 - Here's a general process of using XHR:
1. First we make the XHR Object using the `new XMLHttpRequest()` syntax. You can look at the syntax sugar downstairs after this section.
 2. Then we configure this request. Tell it which place it has to send the request to and what method it has to use. We do this using the `.open()` method.
 - This accepts three arguments, one is the HTTP method it has to send the request in. This could be "POST", "GET", etc. This is optional and if nothing is specified, it'll assume a `GET` request.
 - The second parameter is the URL it needs to send the request to. This is compulsory ofc.
 - The third one is a true or false boolean argument which indicates if the request has to be asynchronous. This is optional and if you leave this empty, the default is true - aka the request will be async.
 3. Then we attach an event listener called `onreadystatechange`. The response once it's sent, has 5 states:
 - 0: uninitialised
 - 1: loading
 - 2: response headers received
 - 3: some response body received
 - 4: request complete
 - To understand this, you should know how a `response` looks like. A `response` is basically just an HTTP transfer
 - A `response` is divided into 2 sections: it's headers and the body.
 - The headers of a response contain information about the response - like the status code, etc. The status code defines what the response is. A status code of 200 means that the response is completely okay. A status code of 404 means that the resource was not found (You'd have seen this in 404 not found pages on the internet)
 - The response body contains the actual content of the response. This could be a JSON response, XML, a blob (images and other resources are sent this way), an HTML document itself, etc. Basically anything you can put in a string, you can put in a response.
 4. When we are ready to send the request, we use the `.send()` method.
 - This takes in only one parameter, the body of your `request`.
 - The structure of the `request` is also the same as the `response` as they both are just HTTP transfers
 - If your method is a `GET`, you don't have to send anything. But if your method is `POST`, then you have to send some information.
 - The `POST` method is used when you have to send some info to the backend server and the server responds with data corresponding to it.
 - For example, if you have to get all the friends of a user on instagram, the post request would contain the username, and the server will respond with the usernames of the user's friends.

Here's how you'd implement these things:

```
<!DOCTYPE html>
<html lang="en">

<head>
  <title>XHR</title>
  <meta charset="UTF-8">
```

HTML

```

<meta name="viewport" content="width=device-width, initial-scale=1">
</head>

<body>
  <div class="information"></div>
  <button>Click to get information</button>
</body>

<script>
  // I will use XHR to fill the div with some information.
  // I will use the GET method to get the information.

  // 1. Create a new XMLHttpRequest object
  let xhr = new XMLHttpRequest();

  // 2. Configure it: GET-request with the URL
  xhr.open('GET', 'https://mocki.io/v1/59303861-55d3-4f27-87fe-71cdb03780a2');
  // this is a mock api that returns a json object with the following information:
  // {
  //   "name": "Anurag Rao",
  //   "age": 18,
  //   "university": "PESU"
  // }
  // you can go to a browser and enter that URL to see the response for yourself

  // 3. Send the request over the network if button is clicked
  let button = document.querySelector('button');
  button.onclick = function () {
    xhr.send(); // no parameters inside because it's a GET function. I don't need to send anything
    to the server.
  }

  // 4. This will be called everytime the ready state changes. from 0 to 1 to 2 to 3 to 4. We only
  need to set the div's content at ready state 4
  xhr.onreadystatechange = function () {
    // verifying that the complete response has loaded and the response is a 200 OK
    if(this.readyState === 4 && this.status === 200) { // show the result
      let data = JSON.parse(xhr.response); // gotta JSON parse because response is a string. This is
      where JSON parse is used.
      console.log(data);
      let information_div = document.querySelector('.information');
      information_div.innerHTML = `Name: ${data.name} <br> Age: ${data.age} <br> University:
      ${data.university}`;
    }
  };
</script>
</html>

```

Before clicking button:

Click to get information

After clicking:

Name: Anurag Rao

Age: 18

University: PESU

Click to get information

- If you think calling that `onreadystatechange()` function on every ready state is kinda a waste of time - that's because it is.
- You can replace it with: `xhr.onload = function () {...}` and it would call only after it's completely loaded. I mentioned the `onreadystatechange` here only because it's on the slides. If you're not doing something different at every stage, it's better to use `.onload`

jQuery AJAX

- You saw how we gotta do so many steps to just send a damn request. jQuery simplifies that. It does the same thing in lesser lines of code. *write less, do more*

`$.ajax()`

- This method accepts a single object as it's parameter. This object contains information about the AJAX request and is called an *options object*
- This options object contains information about the ajax request like `url`, `type` (request type, GET, POST, etc), `datatype` (type of data of the response, can be json, xml, etc) `success` (a callback function that's called on successfully receiving a response) `error` (callback on error)
- Here's an example:

```
$.ajax({  
  url: 'https://api.example.com/data',  
  type: 'GET',  
  dataType: 'json',  
  success: function(data) {  
    console.log(data);  
  },  
  error: function(error) {  
    console.error('Error:', error);  
  }  
});
```

JS

If I wanted to translate our older example to use this, I'd do it like this:

```
<!DOCTYPE html>  
<html lang="en">  
  
<head>  
  <title></title>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1">  
  <link href="css/style.css" rel="stylesheet">  
  <!-- import jquery -->  
  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>  
</head>  
  
<body>  
  <div class="information"></div>  
  <button>Click to get information</button>
```

HTML

```

</body>

<script>
  let button = document.querySelector('button');
  button.onclick = function () {
    $.ajax({
      url: "https://mocki.io/v1/59303861-55d3-4f27-87fe-71cdb03780a2",
      type: 'GET',
      dataType: 'json', // if you set this, you don't need to do JSON.parse
      success: function (data) {
        let text = `Name: ${data.name} <br>Age: ${data.age} <br>University: ${data.university}`;
        $(".information").html(text);
      }
    })
  }
</script>

</html>

```

- We didn't screw with any XHR. jQuery is nice sometimes. Spoiler alert: there's an even better method coming up in the next section

\$.get()

- This and the next `$.post()` and next `$.load()`, `$.getJSON()`, `getScript()` are just derived functions from `$.ajax`.
- They do the same thing but with lesser options and are more specialised, as the name would suggest.
- Since there are lesser options, they do not accept an options object but just normal parameters.
- `$.get` is used to make a `GET` request. It accepts the URL, The data you'd send, and the success function callback.
- This success callback functions accepts the parameters `data`, `textStatus` and `jqXHR`

Here's what each argument represents:

- `data`: The data returned from the server, formatted according to the `dataType` parameter or the data type of the response text. This could be a plain text string, a JSON object, an XML document, etc.
- `textStatus`: A string describing the status of the request. This can be "success", "notmodified", "nocontent", "error", "timeout", "abort", or "parsererror"
- `jqXHR`: A jQuery XMLHttpRequest (jqXHR) object that contains all the details of the completed AJAX request. This object is a superset of the browser's native XMLHttpRequest object. It contains methods and properties that give you more control over the AJAX request and response
- In addition to this, you can also mention a `dataType` attribute to specify what you're expecting from the server. If you leave this blank, jQuery will perform an intelligent guess based on the response headers and other factors.
- How are we sending data if it's a get request? Get requests can kinda cheat in sending data. They can add this thing called URL parameters. These are the values after the `?` in a URL. For example, if I have to send the data `user = anuragrao`, I'd do it like this: `https://api.example.com?user=anuragrao`. The server will read this URL and respond accordingly. This is used when your data you send is small, like just a username.
- Here's how I'd implement the same old example using `.get()`:

```
// HTML ommited for clarity
```

HTML

```

<script>
  let button = document.querySelector('button');
  button.onclick = function () {
    // instead of $.ajax, I use a $.get
    // if this syntax looks scary, it's just a function call but the parameters are in different
    lines.

    $.get(
      // the url
      "https://mocki.io/v1/59303861-55d3-4f27-87fe-71cdb03780a2",

```



```
// the callback
function (data) {
  let text = `Name: ${data.name} <br>Age: ${data.age} <br>University: ${data.university}`;
  $(".information").html(text);
}
);
}
```

\$.post()

- This is similar to `$.get()` but sends a post request.
- The arguments are the same as the `$.get` method.

\$.load()

- This goes one step further in making it easier to load XHR requests. This provides a way to directly *load* the data from a request into an element.
- The response should be of HTML type btw, or how would you put it inside an element
- It takes the parameters URL, Data to send to server, and a callback function that will be called when the response is complete.
- This has to be called from a jQuery object as `$("selector").load()`

I can simplify the example more with this function, here's how we'd do it:

```
// html ommited for clarity
```

HTML

```
<script>
  $(".button").on("click", () => {
    $(".information").load("path/to/url/that/returns/text", () => {
      console.log("Load was performed.");
    })
  })
</script>
```

- This time I used jQuery to select the button as well. I also made use of arrow functions to simplify the callbacks.
- We had two layers callbacks in this function, this takes us back to using promises. Imagine how many layers of callbacks we'd have to use for a more complex operation.

The fetch() API

- This is the modern way of fetching resources from backend servers. We rarely use `XHR` or the `$.ajax()` methods
- The `fetch()` functions is provided by most modern browsers. That's why it's called an API. The terminology is loosely defined, you can call it a function too. Because it is one.
- The syntax of the `fetch()` function is like this:

```
// syntax
const response_promise = fetch("url", {options_object})

// example
fetch("api.example.com",
  { method: 'post',
    headers: { "Content-type": "application/x-www-form-urlencoded; charset=UTF-8"},
    body: 'foo=bar&lorem=ipsum' }
)
```

JS

- You specify if the method, body, etc in the options object. If you leave this empty, it defaults to a GET request.
- The `fetch()` function *returns a promise*. This promise resolves to the response or rejects to an error.
- *It only rejects when there is a network problem*. It does **not** reject on HTTP error like 404 etc. You have to handle that in your `.then()` block.
- Here's how I'd implement the old example using the `fetch()` api:

```

<script>
  let button = document.querySelector('button');
  const resp_promise = fetch("https://mocki.io/v1/59303861-55d3-4f27-87fe-71cdb03780a2");
  button.onclick = function () {
    resp_promise
      .then(
        (resp) => {
          let data = JSON.parse(resp);
          let text = `Name: ${data.name} <br>Age: ${data.age} <br>University: ${data.university}`;
          $(".information").html(text)
        }
      ).catch((error) => {
        console.log("Error: ", error);
      });
  }
</script>

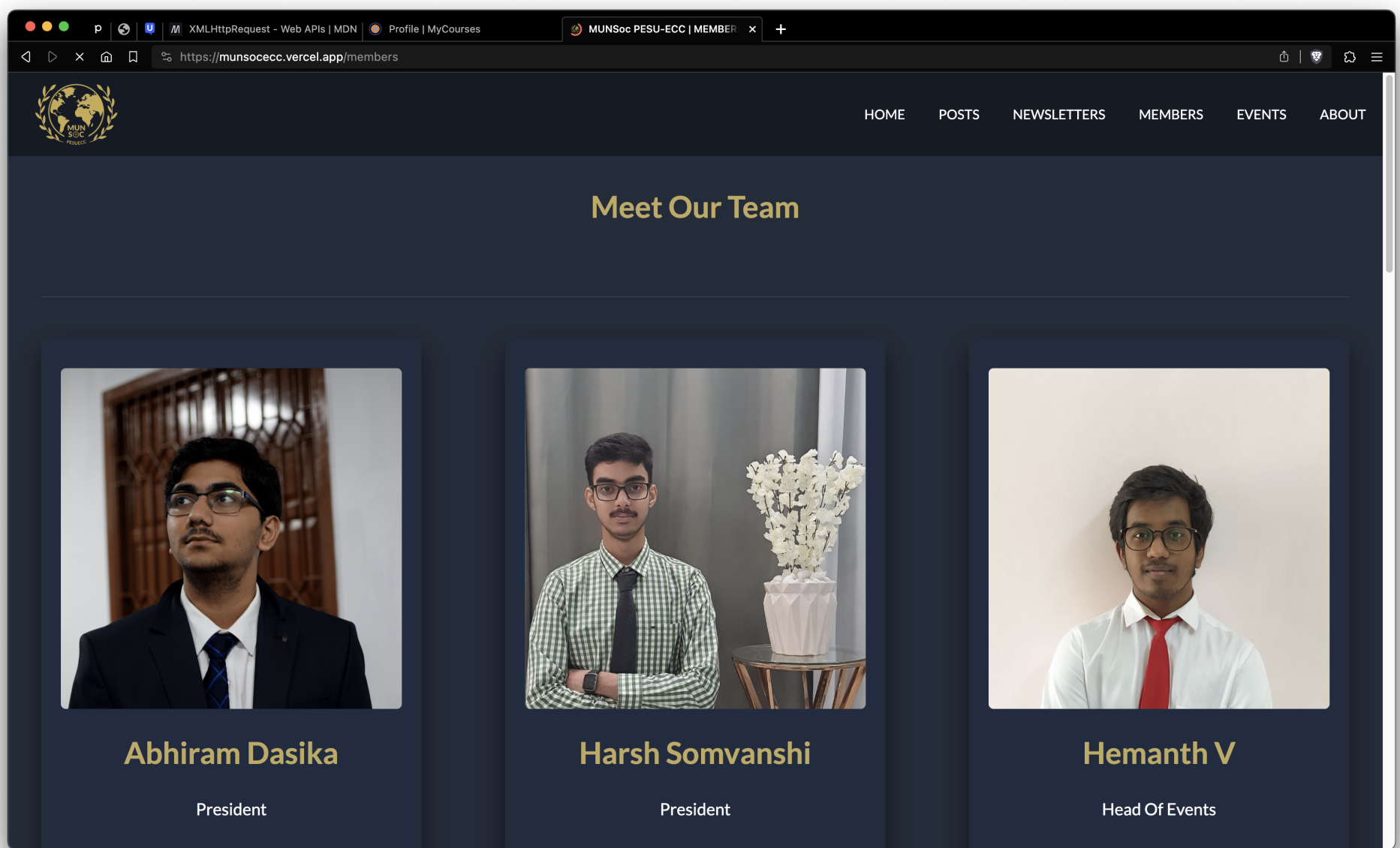
```

Tech Stacks

- Most websites today are a technology sandwich
- What you use to build these websites become ingredients of your sandwich.
- These sandwich ingredients are grouped and these groups are called tech stacks.
- To make your sandwich, you need a frontend, a backend and a database. This is the main structure but some stacks might deviate from this too. Uhh, different kinds of sandwiches
- There are many tech stacks like the **MEAN** stack, the **MERN** stack, the **LAMP** stack, etc.
- **MEAN** = Mongo DB, Express JS, Angular, Node JS
- **MERN** = Mongo DB, Express JS, React, Node JS
- **LAMP** = Linux, Apache, MySQL, PHP
- We'd be doing the MERN Stack in this course. It uses a library called React for it's frontend. Express JS running on Node JS for backend and Mongo DB as a database.
- Express JS helps you make API's to connect your frontend to your database.
- The MongoDB database stores data as JSON objects, rather than in tables like MySQL. These two types of databases are called non relational and relational databases respectively.

React

- React is an open source library developed by the people at Facebook
- It's a declarative library.
- You declare classes for your components
- What's a *component*? To make development easier, you divide each part of your website into reusable components. It's kinda like functions but in UI.
- To make your entire webpage, these components are put together.
- Because of this component architecture, Data can only be passed from a parent component to a child component. Events are propagated from a child component to a parent component.
- For example, imagine you're building a website with multiple cards on it that shows a picture and a text. Something like this:



Every card is a component. This card has three components inside, the image, the name, and the designation.

- The data about what the card must contain can only be passed from the website main page to these cards. These cards can not define data in the other parts of the website.
- If you make an analogy to functions, you can only *pass in* parameters, but can't *return* any value. But, if there's an event, it propagates to the parent components.

Virtual DOM

- A virtual DOM is what react makes to achieve a single page application.
- Manipulating the virtual DOM is really fast.
- Whenever you make changes to your data, this virtual DOM gets re-rendered.
- React compares the changes between the virtual DOM and the real DOM to determine which changes actually matter, and it makes the least amount of DOM changes necessary

Alright Enough Theory. How Do You Create A React App?

- You can either use Node JS and the **npm** package manager. We'll do this in the next unit
- Or we can a CDN like we did in jQuery
- We need two libraries: **React** and **ReactDOM**
- **React** allows you to create reusable components that can be combined to build entire screens, pages, and apps
- **ReactDOM** is responsible for rendering React components to the actual DOM

```
<script src="https://unpkg.com/react@17/umd/react.development.js" crossorigin></script>  
<script src="https://unpkg.com/reactdom@17/umd/reactdom.development.js" crossorigin> </script>
```

HTML

Once we have these two scripts, we can start making components

There are 2 kinds of components: *Stateful* and *Stateless* Components

Stateless Components

- AKA Dumb components
- They are just JS functions that take in **props** and return React Components.
- They do not keep track of *state*. *State* is basically the state of the component. For example, if your component has a counter, your component has to keep track of the state of the counter. So every time it's incremented, it increments from the previous

value.

- This is not possible with a stateless Component. Go for a stateful component

```
function SayHi(props){
  return <h1> Hi! {props.name}</h1>
}
```

Info

- **props** is short for *properties*. These are the properties you pass down to a child.
- Getting the function analogy back, it is similar to passing arguments to this function. These arguments are props.

Stateful Components

- Aka Smart Components
- They are JS classes that extend the **React.Component** class. (Inheritance)
- They can keep track of state.
- The state of a parent component can be passed down to a child component in the form of props to the child.
- These components have properties, just like a class, and have a function called **render()**
- Whatever this **render()** method returns is the component's HTML.

```
class Counter extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      count: 0
    };
  }

  render() {
    return <div>Count is {this.state.count}</div>;
  }
}
```

- There is a better way to manage state called *React Hooks* but that's out of the scope of this unit

Now How Do We Put These In Our Website?

- We use the **ReactDOM.render()** method.
- This takes in two parameters, the component we made, and the element to put this component inside (this should already be in your HTML)

```
<div id="root"></div> //this is the HTML
```

```
// the below is under a script tag
```

```
ReactDOM.render(<Counter />, document.getElementById('root'));
```

You can also use **React.createElement()** to make an element quickly:

```
const element = React.createElement( 'h1', { className: 'greeting' }, 'Hello, World!');
ReactDOM.render(element, document.getElementById("root"))
```

JS

Important

Use this method to make react elements if the question asks you to not use JSX. Questions like these:

- Using React, print welcome to REACTJS on the web page
- Use JSX and NonJSX both

You'd first create an element with `let element = React.createElement("div", null, "welcome to React")` and then use `ReactDOM.render(element, document.getElementById("root"))`

More about what JSX is in the below section

- and in that thoda code above, where we did

```
ReactDOM.render(<Counter />, document.getElementById('root'));
```

There, `<Counter />` is equivalent to `React.createElement(Counter, null, null);`

🔥 Important

All React Components names must start with a capital letter. React will yell at you if you don't follow this rule.

JSX

- We saw until now in React that we put HTML text inside Javascript.
- It's a special version of JS called JSX. You might have also seen that at the top left of code blocks
- JSX allows you to put HTML in your JS. That's why it's called Javascript and XML
- We need a special compiler called babel to interpret JSX. Include it in your script with:

```
<script src="https://unpkg.com/babel-standalone@6.15.0/babel.min.js"></script>
```

HTML

And in your `<script>` tag, you gotta mention `type="text/babel"`. If not, it'll default to normal JS. So the entire script tag for `ReactDOM.render` will be:

```
<script type="text/babel">
  ReactDOM.render(<Counter />, document.getElementById('root'));
</script>
```

HTML

- There are a few differences between JSX elements and HTML though:
 1. All elements MUST be closed. So things like `<br>` and `<img>` will not work. They have to have a close tag or be self closing like `<br />` and `<img />`
 2. The word `class` is a reserved keyword in JS and means something different than in HTML. So `class` is renamed to `className` in your elements. So,

```
<div class = "xyz"></div>
```

HTML

is

```
<div className="xyz"></div>
```

in JSX

3. You can have only one root element in JSX when you're returning a component. You can put multiple elements under this root but only one root element:

```
// NOT ALLOWED
return (
  <div>div1</div>
```



```

<div>div1</div>
<div>div1</div>
)

// Absolutely Fine
return (
  <div>
    <div>div1</div>
    <div>div1</div>
    <div>div1</div>
  </div>
)

```

How Do You Pass Props

The props are passed to the constructor and are available with `this.props`

For example, if we take the SayHi component, and pass props to it about who to say hi to:

```

class SayHi extends React.Component{
  render() {
    return <div>Hi, {this.props.name}</div>;
  }
}
// you have to extend React.component if you're using props, even if it's a stateless component

// you can avoid this by making this stateless component a function:

// The below is equivalent to above and lighter on system resources too
function SayHi(props){
  return <h1> Hi, {props.name}</h1>
}

```

Now when we do ReactDOM.render on this component:

```

ReactDOM.render(
  <SayHi name="cupcake" />,
  document.getElementById("root"); // some element with id root is the container where this SayHi
  element will be put inside.
)

```

You can pass in more than one props too. They look like just HTML attributes.

Prop Types

- By default, these props can be of any datatype. This can lead to potential bugs in the future.
- You can define the datatype of props from a a selection of these datatypes:
- `React.PropTypes.string`: expects a string value.
- `React.PropTypes.number`: expects a numeric value.
- `React.PropTypes.bool`: expects a boolean value.
- `React.PropTypes.array`: expects an array value.
- `React.PropTypes.object`: expects an object value.
- `React.PropTypes.func`: expects a function value.
- `React.PropTypes.element`: expects a React element.

For example, if I wanted to make my `name` prop only accept a string, I'd do:

```

SayName.propTypes = {
  name: React.PropTypes.string,
}

```

Can You Define Default Values For These Props?

- This is useful when no props are passed in to the component and you want to have a default option fallback.
- You can do that using two methods:

Method 1:

```
SayName.defaultProps = {  
  name: "anurag"  
}  
  
class SayHi extends React.Component{  
  render() {  
    return <div>Hi, {this.props.name}</div>;  
  }  
}
```

Method 2: using the `getDefaultProps()` method

- You define a function called `getDefaultProps()` inside the component class. This function can make any logic but it must ultimately return an object with the default props. Here's an example for the SayName component:

```
class SayHi extends React.Component{  
  getDefaultProps(){  
    // logic here if you want  
    // you also can't use this.props here  
    return { // must return an object  
      name: "anurag"  
    }  
  }  
  render() {  
    return <div>Hi, {this.props.name}</div>;  
  }  
}
```

How Do You Transfer Props To A Child Component Inside A Component?

- The old method to do this was to use the `transferPropsTo()` method. However, this has been deprecated and no longer used.
- We use the `...` operator now. This is called a spread operator.
- It's used to deconstruct an object. For example, if our object is:

```
let person = {  
  name: "anurag",  
  age: 18  
}
```

JS

and I do `...person`, it will translate to removing the flower braces and deconstructs to `name = "anurag" age = 18`

- This is mainly used while passing function parameters or props. This was explained better in unit 1, you can go have a look at that too.

Now when we transfer props from a parent to a child component, we do it like this:

```
class SayHi extends React.Component{  
  render() {  
    return (  
      <div> // root wrapper element because I can't return 2 elements  
        <div>Hi, {this.props.name}</div>  
        <ChildComponent prop1="something" prop2="somethingelse" ...this.props />  
      </div>  
    )  
  }  
}
```

```
    );  
  }  
}
```

In this example, we pass in `prop1` and `prop2` to our child component (that is a component defined outside)

In addition to that, we use the spread operator on `this.props` to pass the parent's (`SayHi`) props to `ChildComponent` too

What If You Want To Have Grand Children

- Imagine we use the SayHi Component but we also want to put some other component inside it too.
- How do we specify in `SayHi` where it's children go?
- In other words, if the child `SayHi` also has children.
- We can access the children using `props.children`

```
class Child{  
  render(){  
    return (  
      <div>  
        <h1>{this.props.title}</h1>  
        <div>{this.props.children}</div> // grandchildren go inside this div  
      </div>  
    );  
  }  
};  
  
class Parent{  
  render(){  
    return (  
      <Child title="child">  
        <p>This is some content passed as grand children</p>  
        <button>Click me</button>  
      </Child>  
    );  
  }  
};
```

Styling

- Styles are passed to JSX components as objects.
- These objects contain the property:value as a key-value pair
- These properties must be in camelCase as JS will assume dash (-) is a minus if we use a property like `background-color`. It is translated to `backgroundColor`
- Here's how you style components in JSX:

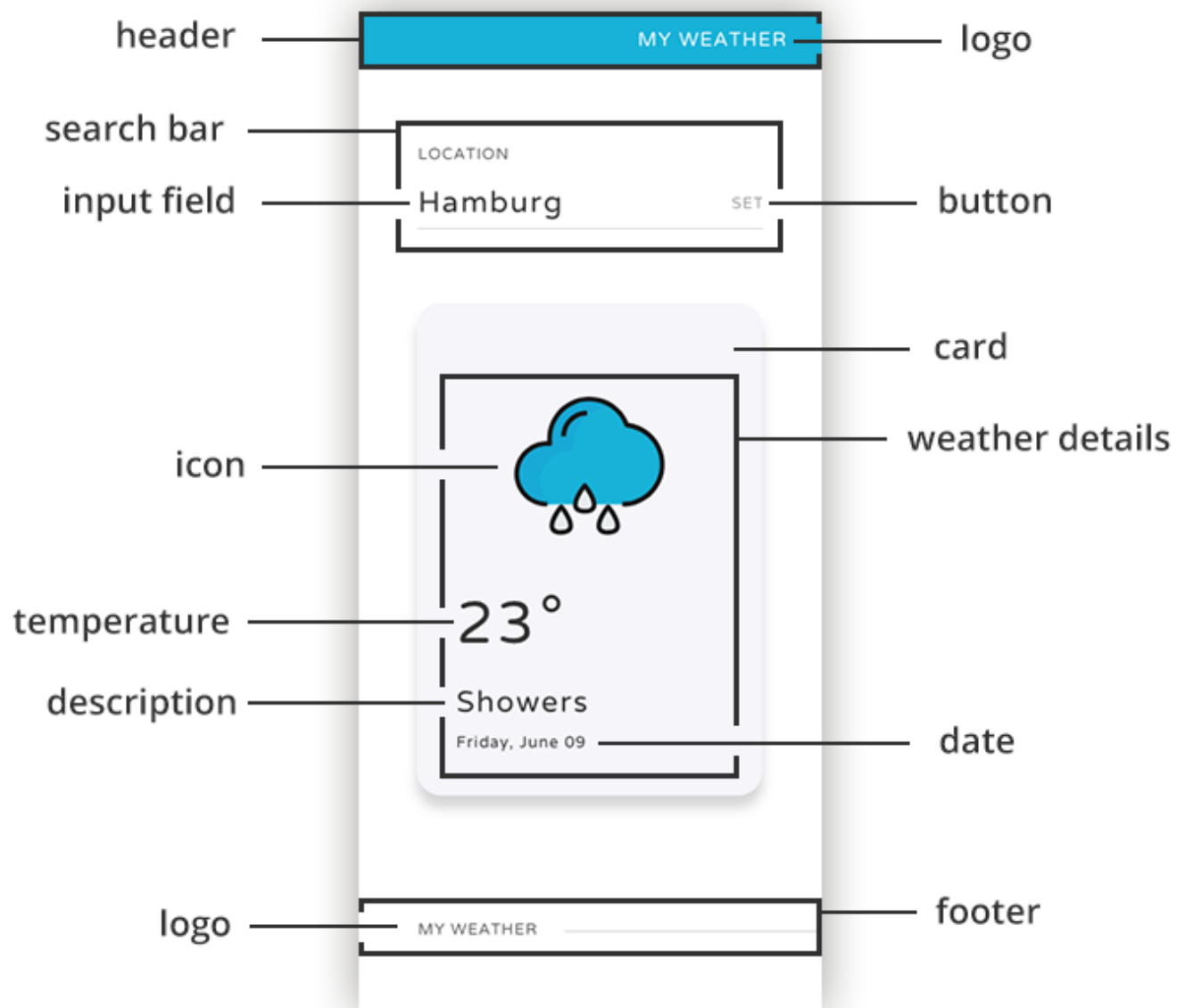
```
let styleObject = {  
  backgroundColor: "black",  
  padding: "10",  
  textAlign: "center",  
}  
<h1 style={styleObject} >Hi! </div>
```

- You can also have them inline. They appear as having 2 flower brackets. One for the `style={}` bracket and one bracket for the object.

```
<h1 style={{backgroundColor: "black"}}>Hi! </div>
```

How To Plan Components

- You plan components the same way you plan functions
- If you want your UI to look like this:



Simple React.js weather app

konstantin münster // konstantin.digital

- You'd have the header section as a component
- The card is another component with weather details component inside of it.
- You could also divide weather details component to have icon, temperature and description components inside of them
- Finally, footer will also be a component
- How much you break stuff down into components depends on your team and how simple you want to make it. Just make sure to have fun at the end. ATB for web tech!